



**TRIBHUVAN UNIVERSITY  
INSTITUTE OF ENGINEERING  
PURWANCHAL CAMPUS**

**A MINOR PROJECT REPORT ON  
AI-IOT BASED SMART ROOM ENERGY MANAGEMENT: A  
COMPARATIVE ANALYSIS OF NAIVE BAYES AND DECISION  
TREE ALGORITHMS ON EMBEDDED HARDWARE**

**BY**

**KARUNA SHRESTHA (080BEI019)  
SAUBHAGYA RAJBHANDARI (080BEI039)  
MUKESHWAR MANDAL (080BEI023)  
SASHANK KHATIWADA (080BEI038)**

**DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING  
PURWANCHAL CAMPUS  
DHARAN, NEPAL**

**AUGUST, 2026**

# **AI-IOT BASED SMART ROOM ENERGY MANAGEMENT SYSTEM**

By

KARUNA SHRESTHA (080BEI019)

SAUBHAGYA RAJBHANDARI (080BEI039)

MUKESHWAR MANDAL (080BEI023)

SASHANK KHATIWADA (080BEI038)

Project Supervisor

Under the supervision of Electronics Computer Department

A project submitted to the Department of Electronics and Computer Engineering in  
partial fulfillment of the requirements for the Bachelor's Degree in Computer  
Engineering

Department of Electronics and Computer Engineering  
Purwanchal Campus, Institute of Engineering  
Tribhuvan University  
Dharan, Nepal

August, 2026

## **ACKNOWLEDGEMENT**

We would like to express our sincere gratitude to the Department of Electronics and Computer Engineering, Purwanchal Campus, for providing the opportunity and facilities to carry out this minor project. We also acknowledge our Head of Department, Er. Manoj Kumar Guragain, for departmental support. We thank our classmates and peers for their constructive feedback during the development and evaluation of the system

## ABSTRACT

A significant amount of electricity in buildings is used by appliances such as fans and lights, and some of this energy is wasted when they operate unnecessarily. Conventional automation systems often use fixed thresholds or simple rule-based control, which may not consider multiple room conditions together. In this project, a Smart Room Automation System is developed using an ESP32, sensors, machine learning, and IoT communication. Two IR sensors placed at the entrance are used to estimate the number of people entering and leaving the room. Temperature and humidity are measured using a DHT11, while two LDRs are used to determine the difference in light level between the inside and outside of the room. An analog gas sensor provides an estimated CO<sub>2</sub> value. These parameters are used by two machine learning models, Decision Tree and Gaussian Naive Bayes, to predict fan speed and light intensity in five levels from 0 to 4. The models were trained and compared using a dataset prepared in Python with scikit-learn. The Decision Tree achieved 81.25% accuracy for fan speed prediction and 86.00% for light intensity prediction, while Naive Bayes achieved 69.44% and 70.88%, respectively. Although the Decision Tree achieved higher accuracy, Naive Bayes was selected for deployment on the ESP32 because of its considerably smaller model size and lower memory footprint, making it more suitable for the resource-constrained microcontroller, allowing the prediction to be performed locally without a cloud-based AI service. The ESP32 sends sensor and actuator data through Wi-Fi using MQTT to a Flutter-based web application. The application displays the sensor values and provides automatic and manual control for the fan and light. In automatic mode, the AI determines the actuator levels, while manual mode allows the user to adjust them using sliders. The project demonstrates a low-cost approach to combining IoT and machine learning for room automation. **Keywords:** Internet of Things, Machine Learning, Smart Room Automation, ESP32, Decision Tree, Naive Bayes, MQTT, Occupancy Sensing, Flutter.

# TABLE OF CONTENTS

|                                                                       |             |
|-----------------------------------------------------------------------|-------------|
| <b>ACKNOWLEDGEMENT</b>                                                | <b>i</b>    |
| <b>ABSTRACT</b>                                                       | <b>ii</b>   |
| <b>LIST OF FIGURES</b>                                                | <b>vi</b>   |
| <b>LIST OF TABLES</b>                                                 | <b>vii</b>  |
| <b>LIST OF ABBREVIATIONS</b>                                          | <b>viii</b> |
| <b>1 INTRODUCTION</b>                                                 | <b>1</b>    |
| 1.1 Background . . . . .                                              | 1           |
| 1.2 Problem Statement . . . . .                                       | 2           |
| 1.3 Motivation . . . . .                                              | 2           |
| 1.4 Objectives . . . . .                                              | 3           |
| <b>2 LITERATURE REVIEW</b>                                            | <b>4</b>    |
| 2.1 Occupancy and Environment Sensing with Machine Learning . . . . . | 4           |
| 2.2 Comparison of Decision Tree and Naive Bayes . . . . .             | 4           |
| 2.3 Existing ESP32-Based Fan and Light Control Projects . . . . .     | 5           |
| 2.4 On-Device Machine Learning and TinyML . . . . .                   | 5           |
| 2.5 Machine Learning for Building Energy Management . . . . .         | 5           |
| 2.6 Summary and Positioning of the Project . . . . .                  | 6           |
| <b>3 METHODOLOGY</b>                                                  | <b>7</b>    |
| 3.1 Overview . . . . .                                                | 7           |
| 3.2 Hardware Design . . . . .                                         | 7           |
| 3.2.1 Components Used . . . . .                                       | 7           |

|          |                                              |           |
|----------|----------------------------------------------|-----------|
| 3.2.2    | Pin Configuration . . . . .                  | 8         |
| 3.2.3    | Occupancy Counting Logic . . . . .           | 8         |
| 3.3      | Dataset and Feature Engineering . . . . .    | 9         |
| 3.4      | Machine Learning Model Development . . . . . | 10        |
| 3.4.1    | Training Pipeline . . . . .                  | 10        |
| 3.4.2    | Selected Hyperparameters . . . . .           | 10        |
| 3.5      | On-Device Model Deployment . . . . .         | 11        |
| 3.6      | Firmware Logic . . . . .                     | 12        |
| 3.7      | Communication Layer . . . . .                | 14        |
| 3.8      | Mobile Dashboard Application . . . . .       | 15        |
| <b>4</b> | <b>RESULTS AND DISCUSSION</b>                | <b>16</b> |
| 4.1      | Model Performance Comparison . . . . .       | 16        |
| 4.2      | Decision Tree Structure . . . . .            | 17        |
| 4.3      | Feature Importance . . . . .                 | 18        |
| 4.4      | Inference Time Comparison . . . . .          | 19        |
| 4.5      | Model Size and Resource Efficiency . . . . . | 20        |
| 4.6      | Model Selection . . . . .                    | 21        |
| 4.7      | On-Device Deployment . . . . .               | 22        |
| 4.8      | System Behaviour . . . . .                   | 22        |
| <b>5</b> | <b>CONCLUSION</b>                            | <b>23</b> |
| 5.1      | Conclusion . . . . .                         | 23        |
| 5.2      | Limitations . . . . .                        | 23        |
| 5.3      | Future Work . . . . .                        | 24        |

|                   |           |
|-------------------|-----------|
| <b>REFERENCES</b> | <b>25</b> |
| <b>APPENDIX</b>   | <b>27</b> |

## LIST OF FIGURES

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Figure 3.1: System architecture of the Smart Room Automation System . . .   | 7  |
| Figure 3.2: Firmware control flow on the ESP32 . . . . .                    | 13 |
| Figure 4.1: Decision Tree for fan-speed prediction . . . . .                | 17 |
| Figure 4.2: Decision Tree for light-intensity prediction . . . . .          | 18 |
| Figure 4.3: Feature importance for fan-speed prediction . . . . .           | 18 |
| Figure 4.4: Feature importance for light-intensity prediction . . . . .     | 19 |
| Figure 4.5: Inference time comparison on the development computer . . . . . | 20 |
| Figure 4.6: Serialized model size comparison . . . . .                      | 21 |



## LIST OF TABLES

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Table 3.1: Hardware components and their role in the system . . . . .      | 8  |
| Table 3.2: ESP32 pin assignment . . . . .                                  | 8  |
| Table 3.3: Dataset feature dictionary . . . . .                            | 9  |
| Table 3.4: Selected Decision Tree hyperparameters . . . . .                | 11 |
| Table 4.1: Test-set performance of Decision Tree and Naive Bayes . . . . . | 16 |
| Table 5.1: Sample rows from <code>sensor_dataset.csv</code> . . . . .      | 27 |
| Table 5.2: ESP32 GPIO assignment . . . . .                                 | 30 |

## LIST OF ABBREVIATIONS

|                 |                                       |
|-----------------|---------------------------------------|
| ADC             | : Analog-to-Digital Converter         |
| API             | : Application Programming Interface   |
| CO <sub>2</sub> | : Carbon Dioxide                      |
| CSV             | : Comma-Separated Values              |
| DHT             | : Digital Humidity and Temperature    |
| GPIO            | : General-Purpose Input/Output        |
| IR              | : Infrared                            |
| LDR             | : Light-Dependent Resistor            |
| ML              | : Machine Learning                    |
| MQTT            | : Message Queuing Telemetry Transport |
| MVC             | : Model-View-Controller               |
| NB              | : Naive Bayes                         |
| PWM             | : Pulse-Width Modulation              |
| RAM             | : Random Access Memory                |
| TLS             | : Transport Layer Security            |
| Wi-Fi           | : Wireless Fidelity                   |

# CHAPTER 1

## INTRODUCTION

### 1.1 Background

Buildings consume a significant amount of electricity for lighting, fans, and other appliances. In homes, offices, and institutional buildings, these appliances are often controlled manually or using simple ON/OFF rules. As a result, fans and lights may remain on when a room is empty or operate at a fixed level even when the actual room condition does not require it [1].

The development of the Internet of Things (IoT) has made it easier to monitor and control such systems using low-cost microcontrollers such as the ESP32. These devices can collect data from different sensors and communicate with other devices through Wi-Fi. Many smart-room projects have used this approach to monitor room conditions and control appliances remotely [2, 3].

However, many existing systems still depend on fixed threshold values. For example, a fan may be turned on when the temperature exceeds a particular value. Such a method does not consider several conditions together, such as the number of people in the room, humidity, and air quality. Machine learning can be used to combine these inputs and make a more flexible control decision. Previous studies have shown that models such as Decision Tree and Naive Bayes can be used with environmental sensor data for occupancy-related prediction [4].

Another approach that is useful for embedded systems is TinyML, where a trained machine learning model is deployed directly on a microcontroller. Lightweight models can run on devices with limited computing resources without requiring a cloud server for every prediction [5–7]. The device can still use communication protocols such as MQTT to send sensor data for monitoring and receive control commands [8].

Based on these ideas, this project develops a **Smart Room Automation System** for a single room. An ESP32 collects occupancy, temperature, humidity, light, and an estimated CO<sub>2</sub> value from different sensors. The system uses machine learning to determine the required fan speed and light intensity. The predicted values are then used

to control the actuators. At the same time, the sensor and actuator data are sent through MQTT to a Flutter-based web application, where the user can monitor the room and manually override the automatic control when required.

## 1.2 Problem Statement

From the study of existing smart-room and energy-management projects, the following gaps were identified:

- **Fixed rule-based control:** Many low-cost automation systems use fixed conditions, such as turning on a fan when the temperature exceeds a particular value. Such rules may not work well when several inputs need to be considered together.
- **Dependence on cloud-based processing:** Systems that use cloud-based machine learning require network communication for prediction. This can introduce additional delay and can affect automatic control when the network is unavailable.
- **Limited control levels:** Many simple systems provide only ON/OFF control. They do not provide different fan-speed or light-intensity levels according to the room condition.
- **Limited manual override:** A fully automatic system may not provide an easy way for a user to temporarily adjust the fan or light when the automatic decision is not suitable.

This project attempts to address these gaps by combining occupancy sensing, environmental sensing, lightweight machine learning, on-device prediction, and MQTT-based monitoring with manual control through a Flutter application.

## 1.3 Motivation

The demand for electricity is increasing, while efficient use of available energy is important, particularly in places such as classrooms, offices, and other institutional buildings. The number of people in a room can change frequently, and the required fan speed or light intensity may also change accordingly. Therefore, a system that can consider occupancy and room conditions before controlling appliances can be useful for room automation.

The project also provides practical experience in sensor interfacing, occupancy detection, dataset preparation, machine learning model training, embedded system programming, MQTT communication, and Flutter application development. These areas are closely related to the concepts studied in computer engineering and provide an opportunity to apply them in a single working system.

## **1.4 Objectives**

The main objectives of this project are:

- To design and develop an ESP32-based system for monitoring occupancy, temperature, humidity, light level, and an estimated CO<sub>2</sub> value in a room.
- To train and compare Decision Tree and Naive Bayes models for predicting fan speed and light intensity, and deploy the better-performing and compatible model on the ESP32.
- To develop a Flutter-based web dashboard for real-time monitoring and manual control of the fan and light through MQTT.

## **CHAPTER 2**

### **LITERATURE REVIEW**

This chapter reviews previous work related to occupancy sensing, machine learning-based room automation, ESP32-based control systems, TinyML, and building energy management. The reviewed studies help identify the approaches used in existing systems and the areas that can be improved.

#### **2.1 Occupancy and Environment Sensing with Machine Learning**

Toutiaee [4] studied room occupancy detection using environmental sensor data such as temperature, humidity, light, and CO<sub>2</sub>. Different machine learning algorithms, including Decision Tree, Naive Bayes, Random Forest, and KNN, were evaluated for occupancy detection. The study showed that environmental sensor data can be useful for occupancy-related classification without relying on cameras.

The approach is related to this project because environmental parameters are also used as inputs to machine learning models. However, instead of predicting only whether a room is occupied, this project uses occupancy count along with environmental parameters to predict fan speed and light intensity.

#### **2.2 Comparison of Decision Tree and Naive Bayes**

Yadav and Thareja [9] compared the performance of Naive Bayes and Decision Tree classifiers using different datasets. Their study showed that the performance of the two algorithms can vary depending on the dataset, although Decision Tree performed better in several cases.

Sheth et al. [10] also compared different machine learning classification algorithms and highlighted that no single algorithm is always the best for every type of dataset. These studies support the decision to train and evaluate both Decision Tree and Naive Bayes models in this project instead of selecting a model without testing.

### **2.3 Existing ESP32-Based Fan and Light Control Projects**

ESP32 has been used in several low-cost automation projects for controlling room appliances. Mueller [2] developed an ESP32-based fan controller that uses temperature information to control fan speed and communicates its status using MQTT. Ghader [3] developed an ESP32-based system for controlling lights based on room occupancy using a PIR sensor.

These projects demonstrate the practical use of ESP32 and MQTT for room automation. However, the control logic in these systems is mainly based on individual sensor conditions or simple rules. In comparison, this project combines multiple sensor inputs and uses machine learning to determine the required fan speed and light intensity.

### **2.4 On-Device Machine Learning and TinyML**

TinyML allows machine learning models to be executed on resource-constrained devices instead of relying entirely on a cloud or remote server. Warden and Situnayake [5] describe the use of machine learning models on low-power microcontrollers and discuss the process of deploying trained models on embedded devices.

Sanchez-Iborra and Skarmeta [6] discuss the challenges of deploying machine learning on small embedded devices, including limitations in memory, processing capability, and energy consumption. Dutta and Bharali [7] provide a broader review of TinyML and its combination with IoT systems.

These studies show that lightweight models are suitable for edge-based applications. In this project, a Decision Tree is selected for deployment because its prediction process can be represented using simple comparisons and can therefore be executed directly on the ESP32.

### **2.5 Machine Learning for Building Energy Management**

Talib and Croock [1] investigated the use of machine learning for intelligent energy management in buildings. Their work discusses the use of data-driven approaches to improve energy management compared with conventional control methods.

This provides the broader motivation for applying machine learning to room-level automation. The present project focuses on a smaller scale by using sensor information from a single room to control fan speed and light intensity.

## **2.6 Summary and Positioning of the Project**

The reviewed studies show that IoT devices such as the ESP32 can be effectively used for sensor monitoring and appliance control, while machine learning can be applied to environmental and occupancy-related data. Previous work has also demonstrated the possibility of running lightweight machine learning models directly on embedded devices.

However, the reviewed ESP32-based projects mainly use simple control rules, while machine learning-based studies are generally focused on prediction tasks rather than direct appliance control on a microcontroller. Based on these observations, this project combines both approaches by using multiple sensor inputs, comparing Decision Tree and Naive Bayes models, and deploying the selected Decision Tree model directly on the ESP32.

The system also combines on-device prediction with MQTT-based communication and a Flutter dashboard. This allows the automatic control to operate on the ESP32 while the user can monitor the system and manually control the fan and light when required.



# CHAPTER 3

## METHODOLOGY

### 3.1 Overview

The proposed Smart Room Automation System was developed in four main stages: sensor data collection and occupancy counting, dataset preparation and machine learning model training, deployment of the trained model on the ESP32, and monitoring and control through a Flutter-based mobile application. Figure 3.1 shows the overall architecture of the system.

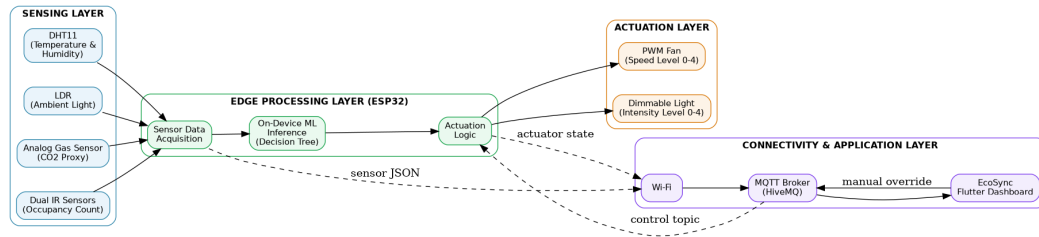


Figure 3.1: System architecture of the Smart Room Automation System

The ESP32 acts as the main controller of the system. It collects sensor readings, maintains the occupancy count, performs machine learning predictions and controls the fan and light. MQTT is used to exchange sensor and actuator information with the mobile application, while the machine learning prediction itself is performed locally on the ESP32.

### 3.2 Hardware Design

#### 3.2.1 Components Used

The main hardware components used in the system and their functions are shown in Table 3.1.

Table 3.1: Hardware components and their role in the system

| Component                | Role                                                 |
|--------------------------|------------------------------------------------------|
| ESP32                    | Processing, Wi-Fi, MQTT and ML inference             |
| DHT11 Sensor             | Temperature and humidity measurement                 |
| Two LDRs                 | Inside and outside light-level comparison            |
| Analog Gas Sensor        | Indicative air-quality / CO <sub>2</sub> proxy value |
| Two IR Sensors           | Direction-based occupancy counting                   |
| L298N Motor Driver       | Driving Fan / LED and control using PWM              |
| 12V DC Fan and LED Strip | Prototype compatible prop                            |

### 3.2.2 Pin Configuration

The ESP32 pin assignments used in the project are shown in Table 3.2.

Table 3.2: ESP32 pin assignment

| Signal        | GPIO Pin | Notes         |
|---------------|----------|---------------|
| IR Sensor 1   | GPIO 23  | Digital input |
| IR Sensor 2   | GPIO 22  | Digital input |
| DHT11 Data    | GPIO 4   | Digital input |
| LDR (inside)  | GPIO 34  | Analog input  |
| LDR (outside) | GPIO 36  | Analog input  |
| Gas Sensor    | GPIO 35  | Analog input  |

The analog sensors were connected to ADC1 pins of the ESP32. This avoids the ADC2 and Wi-Fi resource conflict that can occur when ADC2 pins are used while Wi-Fi is active.

### 3.2.3 Occupancy Counting Logic

Two IR sensors are placed a short distance apart at the entrance of the room. The order in which the sensors are triggered is used to determine whether a person is entering or leaving.

If IR1 is triggered first and IR2 is triggered within the defined time window, the system considers the movement as an entry and increases the occupancy count by one. Similarly, if IR2 is triggered first and IR1 is triggered within the time window, the system considers it an exit and decreases the occupancy count by one. The occupancy count is not allowed to become negative.

The IR sensors are monitored using hardware interrupts. The interrupt routines record the sensor events, while the main program processes the events and updates the occupancy count. A short debounce mechanism is also used to reduce false triggering caused by sensor noise.

### 3.3 Dataset and Feature Engineering

An 8000-row synthetic dataset named `sensor_dataset.csv` was prepared for training the machine learning models. Each row contains the sensor-related input features and the corresponding target values for fan speed and light intensity.

The main input features used by the models are `count`, `co2_level`, `ldr_diff`, `temp`, and `humidity`. The two target variables are `fan_speed` and `light_intensity`, with output levels ranging from 0 to 4.

Table 3.3: Dataset feature dictionary

| Feature                      | Range             | Description                                      |
|------------------------------|-------------------|--------------------------------------------------|
| <code>count</code>           | 0–30              | Occupancy count                                  |
| <code>co2_level</code>       | 200–2500          | Indicative air-quality value                     |
| <code>ldr_diff</code>        | Dataset dependent | Difference between outside and inside LDR values |
| <code>temp</code>            | Dataset dependent | Temperature in °C                                |
| <code>humidity</code>        | Dataset dependent | Relative humidity in %                           |
| <code>fan_speed</code>       | 0–4               | Fan-speed target level                           |
| <code>light_intensity</code> | 0–4               | Light-intensity target level                     |

Two LDRs are used instead of a single sensor so that the system can compare the light level inside the room with the outside light level. The feature used by the machine learning model is calculated as

$$\text{ldr\_diff} = \text{ldrOutside} - \text{ldrInside}.$$

A larger positive difference generally indicates that the outside is brighter than the inside, meaning that additional artificial lighting may be required.

Separate input features are used for the two prediction tasks. The fan-speed model uses `count`, `co2_level`, `temp`, and `humidity`. The light-intensity model uses `count`, `co2_level`, and `ldr_diff`.

### 3.4 Machine Learning Model Development

#### 3.4.1 Training Pipeline

The machine learning models were developed in Python using `pandas` and `scikit-learn` [11]. A Decision Tree classifier was developed as the main candidate model, while Gaussian Naive Bayes was implemented as a comparison model.

The following procedure was used during model development:

1. The dataset was divided into training and testing sets using an 80/20 split with `random_state=42`.
2. A baseline classifier was trained using the training portion of the dataset.
3. `GridSearchCV` with 5-fold cross-validation was used to determine suitable hyperparameters for the classifiers.
4. The best-performing model was retrained using the training data.
5. The final model was evaluated on the separate 20% test set.
6. Accuracy, precision, recall, F1-score and confusion matrices were used to evaluate classification performance.
7. Five-fold cross-validation was also used to obtain an additional estimate of model performance.
8. For the Decision Tree, feature importance was extracted to identify the relative contribution of the input features.

The Decision Tree and Gaussian Naive Bayes models were evaluated using the same dataset and test split so that their performance could be compared fairly.

#### 3.4.2 Selected Hyperparameters

The best Decision Tree hyperparameters obtained using `GridSearchCV` are shown in Table 3.4.

Table 3.4: Selected Decision Tree hyperparameters

| Hyperparameter        | Fan Speed | Light Intensity |
|-----------------------|-----------|-----------------|
| Criterion             | Gini      | Entropy         |
| Maximum depth         | 11        | 7               |
| Minimum samples split | 10        | 2               |
| Minimum samples leaf  | 4         | 8               |

For Gaussian Naive Bayes, the selected `var_smoothing` values were  $1 \times 10^{-6}$  for the fan-speed model and  $3.16 \times 10^{-5}$  for the light-intensity model.

The final comparison between Decision Tree and Gaussian Naive Bayes is presented in Chapter 5.

### 3.5 On-Device Model Deployment

After training and evaluation, the selected Decision Tree models were converted into C++ header files named `fan_speed_model.h` and `light_intensity_model.h`. The conversion follows the Eloquent ML / `micromlgen` approach, where the learned decision nodes are represented using nested `if/else` conditions.

A simplified example of the generated prediction structure is shown below:

```
int predict(float *x) {
    if (x[2] <= 34.45000076293945) {
        if (x[0] <= 13.5) {
            if (x[3] <= 66.95000076293945) {
                ...
                return 0;
            }
        }
    }
    ...
}
```

The generated model does not require a separate machine learning library during prediction. Instead, the ESP32 evaluates the learned comparison rules and returns the predicted class. This makes the Decision Tree practical for local inference on the mi-

crocontroller.

The generated fan-speed model contains 431 leaf nodes and is approximately 138 KB in generated C++ source code. The light-intensity model contains 112 leaf nodes and is approximately 29 KB in generated C++ source code. These values represent the generated source files and should not be interpreted as the exact flash or RAM usage of the final compiled ESP32 firmware.

### **3.6 Firmware Logic**

The ESP32 firmware is organized around the `setup()` and `loop()` functions. Figure 3.2 shows the main control flow.

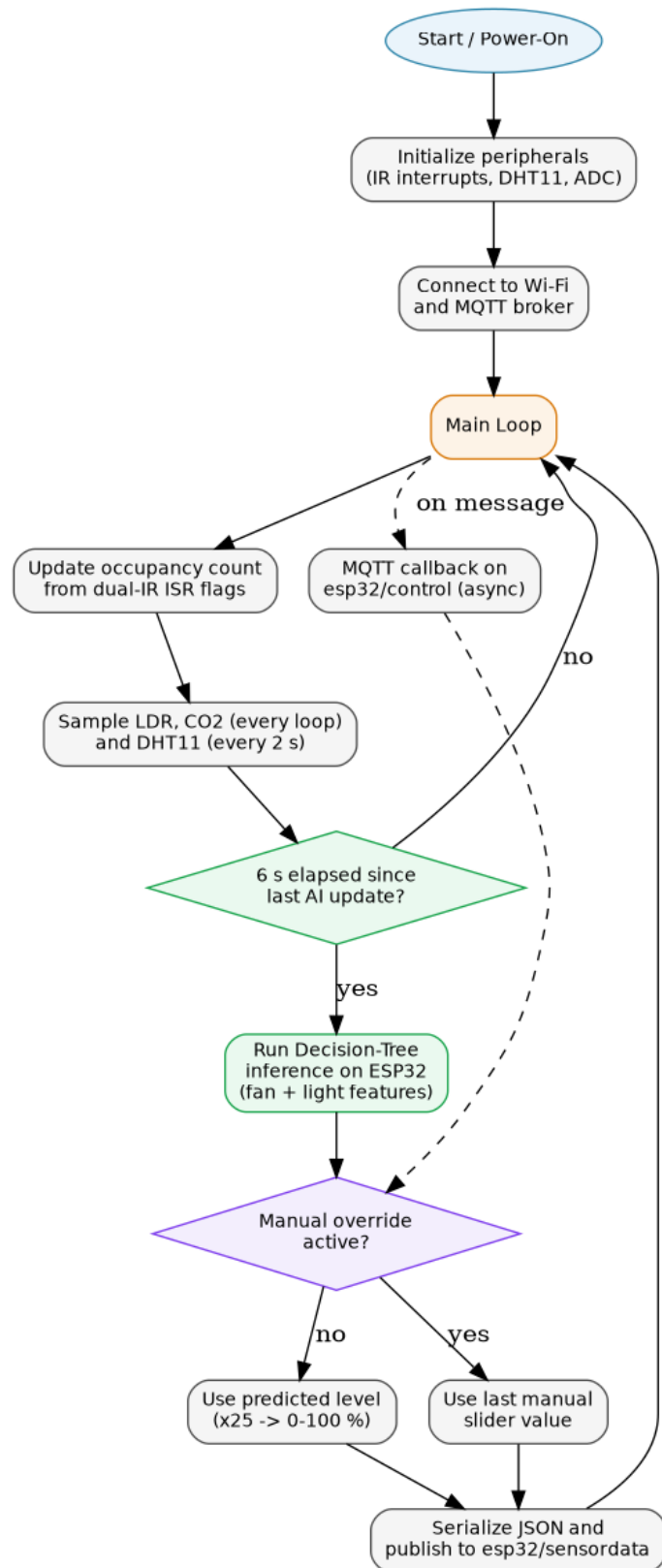


Figure 3.2: Firmware control flow on the ESP32

The main design features of the firmware are:

- **Occupancy detection:** IR sensor events are detected using interrupts, while the main loop processes the events and updates the occupancy count.
- **Sensor sampling:** Different sensors are read at different intervals according to their requirements. The DHT11 is read every 2 seconds, while machine learning prediction and MQTT publishing are performed every 6 seconds.
- **Automatic and manual control:** The fan and light have separate automatic and manual modes. This allows the user to manually control one actuator while leaving the other under automatic control.
- **Connection recovery:** The ESP32 checks the Wi-Fi and MQTT connections and attempts to reconnect when a connection is lost. The local prediction and actuator control can continue while the network connection is temporarily unavailable.

### 3.7 Communication Layer

The ESP32 sends sensor readings and the current actuator states to the MQTT broker using the `esp32/sensordata` topic. The information is transmitted in JSON format. An example message is:

```
{
  "count": 3,
  "temp": 28.4,
  "humidity": 61.2,
  "ldr": 210,
  "co2": 950,
  "fan_speed": 50,
  "light_intensity": 75,
  "fan_manual": false,
  "light_manual": false
}
```

The ESP32 also subscribes to the `esp32/control` topic to receive commands from the Flutter application. These commands can be used to change the fan or light level and to switch between automatic and manual modes.



During development and testing, the public HiveMQ MQTT broker was used on port 1883. This was suitable for the prototype and testing stage. A private and authenticated MQTT broker would be more appropriate for a practical deployment.

### 3.8 Mobile Dashboard Application

The Flutter application, is used for monitoring the sensor readings and controlling the actuators. The application follows a simple MVC-style structure.

- **Models:** `SensorReading` and `ActuatorState` convert MQTT JSON messages into Dart objects.
- **Services:** `MqttService` manages the MQTT connection and provides incoming messages through Dart streams.
- **Controller:** `DashboardController` maintains the application state and sends control commands to the ESP32.
- **Views:** `EcoSyncDashboard`, `MetricCard`, and `ActuatorCard` display sensor values and provide controls for the fan and light.

The dashboard receives sensor and actuator information from the ESP32 through MQTT. In automatic mode, the application displays the values determined by the ESP32. In manual mode, the user can adjust the fan speed or light intensity using the available controls.

The application therefore acts mainly as a monitoring and user-control interface, while the actual sensor processing, machine learning prediction and actuator control remain on the ESP32.

## CHAPTER 4

### RESULTS AND DISCUSSION

This chapter presents the results obtained from the machine learning models developed for the Smart Room Automation System. A Decision Tree and a Gaussian Naive Bayes classifier were trained for two separate tasks: predicting fan speed and predicting light intensity. Both models were trained using the same dataset and evaluated using the same 20% test set.

The main classification results are presented in Table 4.1. Based on these results, together with the inference-time and model-size comparison, the Decision Tree was selected for deployment on the ESP32.

#### 4.1 Model Performance Comparison

The Decision Tree and Gaussian Naive Bayes models were evaluated using accuracy, weighted precision, weighted recall and weighted F1-score. The results for both prediction tasks are shown in Table 4.1.

Table 4.1: Test-set performance of Decision Tree and Naive Bayes

| Target          | Model         | Accuracy | Precision | Recall | F1-Score |
|-----------------|---------------|----------|-----------|--------|----------|
| fan_speed       | Decision Tree | 81.25%   | 83.20%    | 79.67% | 81.29%   |
| fan_speed       | Naive Bayes   | 69.44%   | 70.55%    | 65.38% | 64.11%   |
| light_intensity | Decision Tree | 86.00%   | 85.51%    | 86.43% | 85.96%   |
| light_intensity | Naive Bayes   | 70.88%   | 75.17%    | 69.81% | 71.70%   |

For fan-speed prediction, the Decision Tree achieved an accuracy of 81.25%, compared with 69.44% for Naive Bayes. It also achieved higher precision and F1-score, with values of 83.20% and 81.29%, respectively.

For light-intensity prediction, the Decision Tree achieved 86.00% accuracy, 85.51% precision and 85.96% F1-score. Naive Bayes achieved 70.88% accuracy, 75.17% precision and 71.70% F1-score.

Overall, the Decision Tree performed better than Naive Bayes for both prediction tasks. The accuracy improvement was 11.81 percentage points for fan-speed prediction and

15.12 percentage points for light-intensity prediction.

The better performance of the Decision Tree is reasonable because the sensor features can affect the output together. For example, fan speed can depend on a combination of occupancy, temperature and humidity. Naive Bayes assumes conditional independence between the input features, while the Decision Tree can learn combinations of features through its decision rules.

## 4.2 Decision Tree Structure

Separate Decision Tree models were trained for fan-speed and light-intensity prediction. The two models have different structures because the two outputs depend on different sensor conditions.

The fan-speed model uses a maximum depth of 11, while the light-intensity model uses a maximum depth of 7. The generated trees are shown in Figures 4.1 and 4.2.

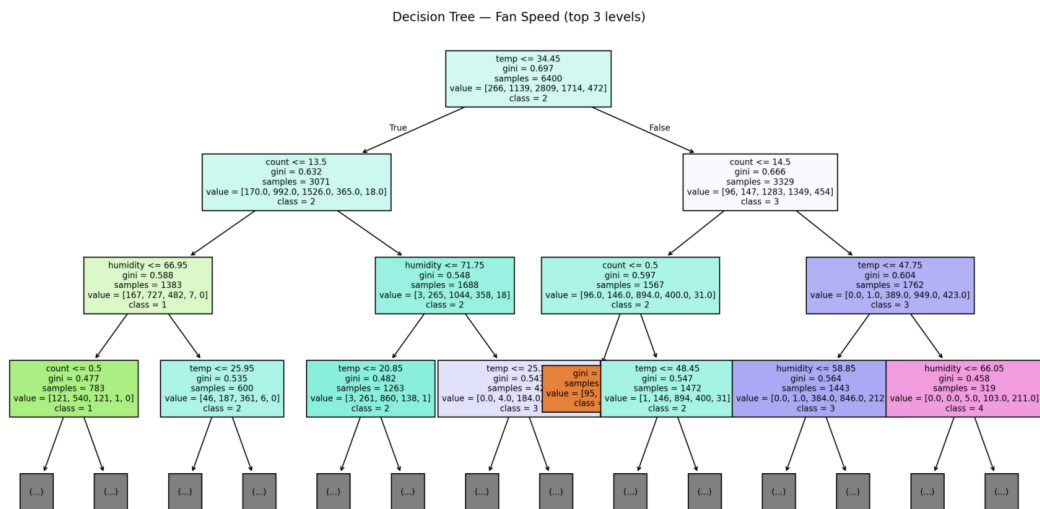


Figure 4.1: Decision Tree for fan-speed prediction

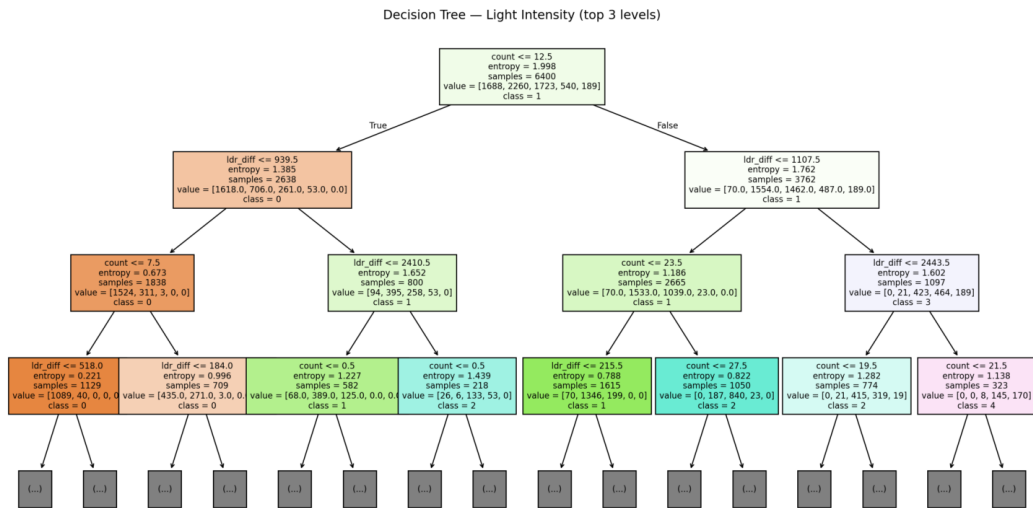


Figure 4.2: Decision Tree for light-intensity prediction

The tree models consist mainly of simple comparisons between sensor values and learned threshold values. This structure makes them suitable for conversion into C++ code and deployment on the ESP32.

### 4.3 Feature Importance

Feature importance was obtained from the trained Decision Tree models to identify the sensor features that contributed most to the predictions.

The feature importance results for fan-speed and light-intensity prediction are shown in Figures 4.3 and 4.4.

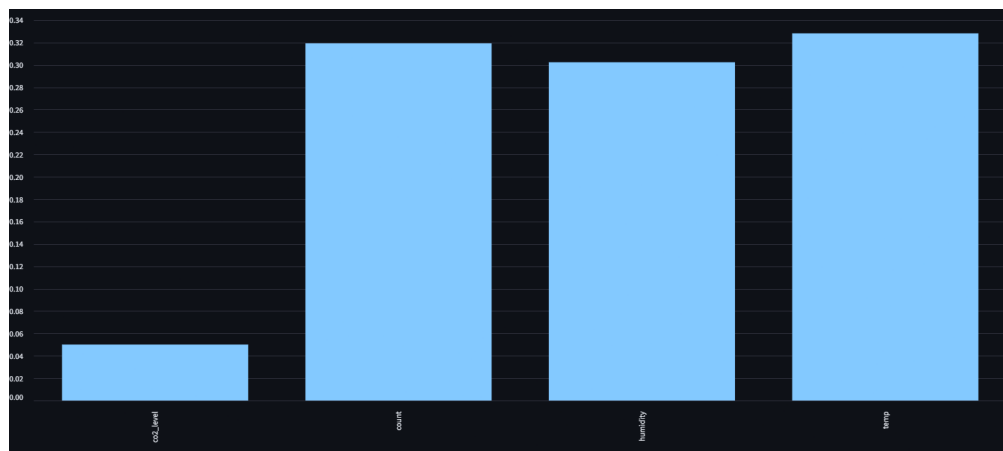


Figure 4.3: Feature importance for fan-speed prediction

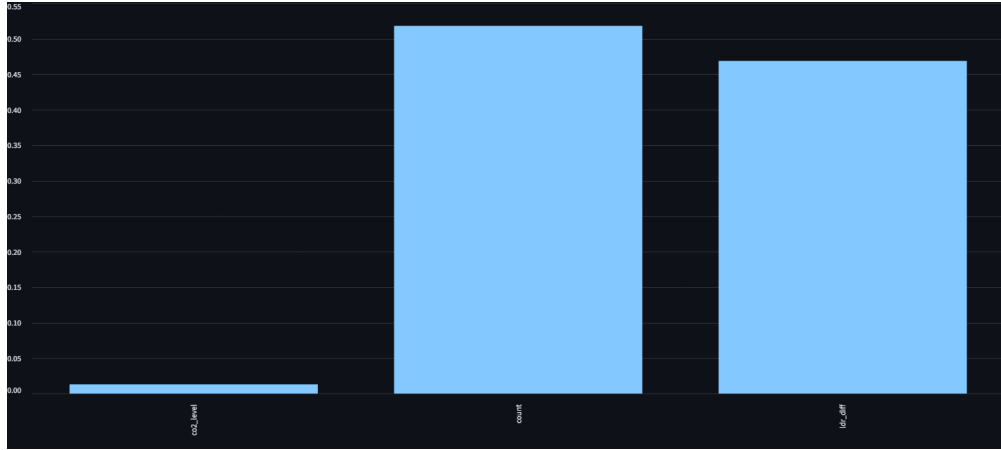


Figure 4.4: Feature importance for light-intensity prediction

For fan-speed prediction, temperature, occupancy count and humidity were the most important features, while the CO2 value had a smaller contribution. For light-intensity prediction, occupancy count and the LDR difference were the main contributing features.

These results describe the behaviour of the trained dataset and should not be considered a general statement about sensor importance in every room.

#### 4.4 Inference Time Comparison

Inference time was measured during the model comparison on the development computer. The Decision Tree required 0.4491 ms for fan-speed prediction and 0.4372 ms for light-intensity prediction. The corresponding Naive Bayes times were 0.5783 ms and 0.4514 ms.

The measured inference-time comparison is shown in Figure 4.5.

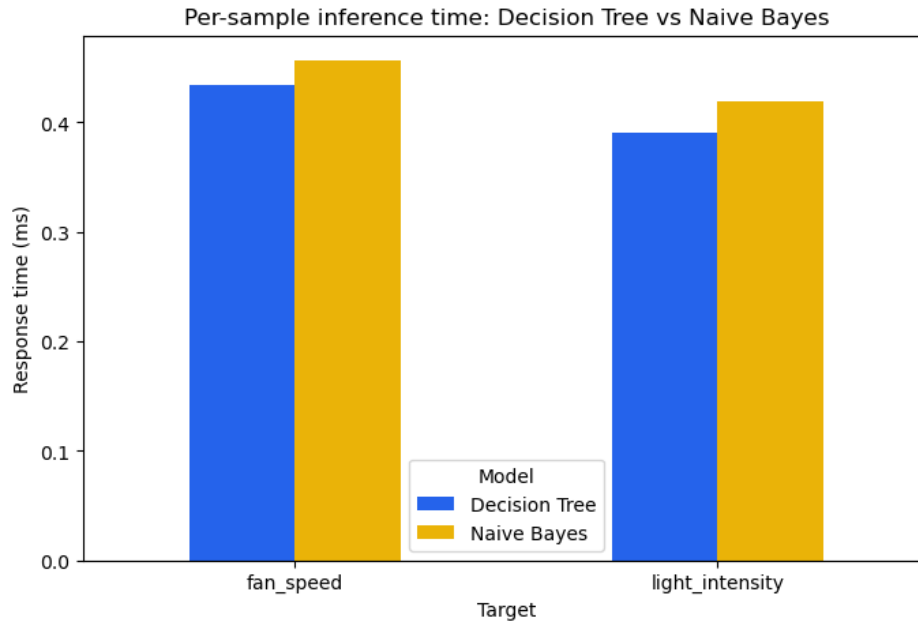


Figure 4.5: Inference time comparison on the development computer

The Decision Tree was slightly faster than Naive Bayes in both measurements. However, the difference was relatively small, particularly for the light-intensity model. Therefore, inference time alone was not considered sufficient to select the final model.

These measurements were obtained on the development computer and should not be considered as the actual inference time of the ESP32. A hardware benchmark using `micros()` around the `predict()` function would be required to measure the actual ESP32 inference time.

#### 4.5 Model Size and Resource Efficiency

Model size was also compared between the two classifiers. The serialized Decision Tree models had sizes of 36.74 KB for fan-speed prediction and 29.83 KB for light-intensity prediction. The corresponding Naive Bayes models were much smaller, with sizes of 1.11 KB and 1.03 KB, respectively.

The model-size comparison is shown in Figure 4.6.

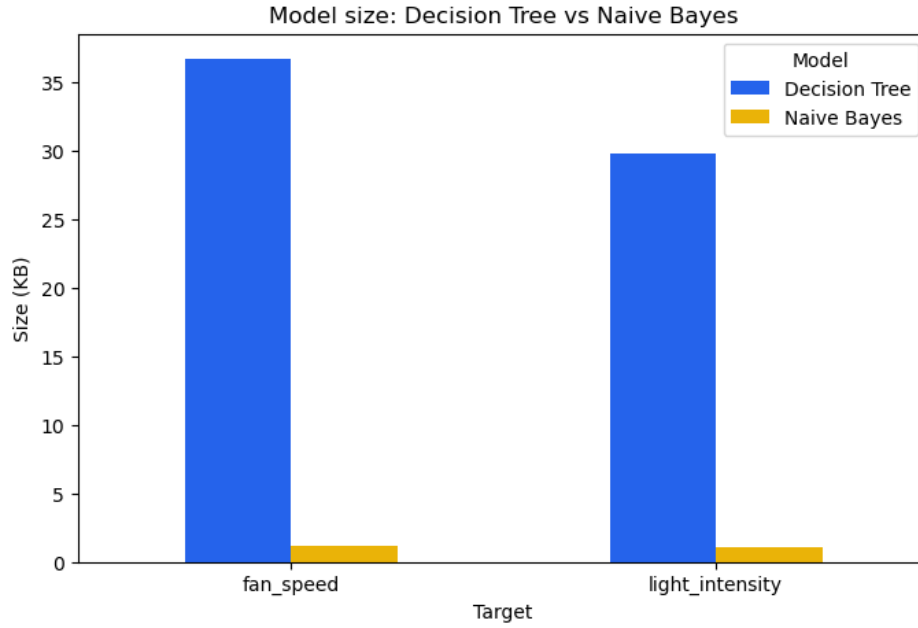


Figure 4.6: Serialized model size comparison

Naive Bayes is clearly more compact than the Decision Tree. This is an advantage for embedded systems where memory resources are limited. However, the serialized model size shown here should not be considered the exact flash or RAM usage of the final ESP32 firmware. The actual embedded footprint depends on the generated C++ code, compiler optimization and the other libraries included in the firmware.

#### 4.6 Model Selection

The Decision Tree was selected for the final implementation because it provided better classification performance for both prediction tasks.

For fan-speed prediction, it achieved 81.25% accuracy compared with 69.44% for Naive Bayes. For light-intensity prediction, it achieved 86.00% compared with 70.88%.

Although Naive Bayes produced smaller models, the higher prediction performance of the Decision Tree was considered more important for the proposed application. The Decision Tree also showed slightly lower inference times in the development-computer measurements.

Therefore, the two Decision Tree models were selected for deployment on the ESP32.

## **4.7 On-Device Deployment**

The selected Decision Tree models were converted into C++ header files and included directly in the ESP32 firmware. The fan-speed and light-intensity predictions are performed locally by the microcontroller.

The ESP32 reads the sensor values, calculates the required features and passes them to the corresponding Decision Tree. The predicted output is then used for fan and light control. MQTT is used for monitoring and manual control through the EcoSync Flutter application.

The exact ESP32 inference time and RAM usage were not measured during this project. Therefore, no specific ESP32 inference-time or RAM value is claimed.

## **4.8 System Behaviour**

In automatic mode, the ESP32 uses the Decision Tree predictions to determine the fan speed and light intensity. In manual mode, the user can control the corresponding actuator through the EcoSync dashboard.

The basic prediction and actuator control are performed locally on the ESP32, so they do not depend on the MQTT connection. MQTT is mainly used for communication between the ESP32 and the EcoSync application.

Actual energy savings were not measured during this project. Therefore, the energy-saving performance of the system would require longer-term testing in a real room.



## CHAPTER 5

### CONCLUSION

#### 5.1 Conclusion

This project designed and implemented a Smart Room Automation System using an ESP32, environmental sensors, machine learning and an MQTT-based mobile dashboard. The system uses occupancy, temperature, humidity, light and indicative CO<sub>2</sub> data to automatically control fan speed and light intensity.

Decision Tree and Gaussian Naive Bayes classifiers were trained and compared using the same dataset. The Decision Tree performed better for both tasks, achieving 81.25% accuracy for fan-speed prediction and 86.00% for light-intensity prediction, compared with 69.44% and 70.88% for Naive Bayes. Although Naive Bayes required less model storage, the better prediction performance of the Decision Tree made it more suitable for this application.

The trained Decision Tree models were converted to C++ and deployed on the ESP32, allowing prediction and actuator control to be performed locally without depending on cloud-based machine learning. MQTT and the EcoSync Flutter application provide monitoring and manual control of the system.

Overall, the project demonstrates that a low-cost ESP32 can be used to implement a practical machine-learning-based room automation prototype.

#### 5.2 Limitations

- The models were trained using a synthetic dataset rather than long-term real-room data.
- The gas sensor provides only an indicative CO<sub>2</sub>/air-quality value and is not calibrated.
- Inference time and RAM usage were measured on the development computer, not directly on the ESP32.
- Actual energy savings were not measured during the project.

- A public MQTT broker was used for development and testing rather than a secured private broker.

### **5.3 Future Work**

- Collect real sensor data from an occupied room and retrain the models.
- Measure actual ESP32 inference time, RAM usage and power consumption.
- Measure and compare energy consumption with manual and threshold-based control.
- Replace the uncalibrated gas sensor with a dedicated NDIR CO<sub>2</sub> sensor.
- Use a private MQTT broker with authentication and TLS for a real deployment.
- Extend the system to multiple rooms and add energy monitoring to the EcoSync dashboard.

## REFERENCES

- [1] M. M. Talib and M. S. Croock, “Optimizing energy consumption in buildings: Intelligent power management through machine learning,” *Mathematical Modelling of Engineering Problems*, vol. 11, no. 3, pp. 765–772, 2024.
- [2] K. Mueller, “Esp32 fan controller with temperature sensor and mqtt,” 2024. [Online]. Available: <https://github.com/KlausMu/esp32-fan-controller>
- [3] O. Ghader, “Control lights based on room occupancy using pir and esp32,” 2025. [Online]. Available: <https://omarghader.github.io/control-lights-room-occupancy-pir-esp32/>
- [4] M. Toutiaee, “Occupancy detection in room using sensor data,” *arXiv preprint arXiv:2101.03616*, 2021.
- [5] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*, 1st ed. O’Reilly Media, 2020.
- [6] R. Sanchez-Iborra and A. F. Skarmeta, “Tinymml-enabled frugal smart objects: Challenges and opportunities,” *IEEE Circuits and Systems Magazine*, vol. 20, no. 3, pp. 4–18, 2020.
- [7] L. Dutta and S. Bharali, “Tinymml meets iot: A comprehensive survey,” *Internet of Things*, vol. 16, p. 100461, 2021.
- [8] OASIS, “Mqtt version 5.0 oasis standard,” 2019. [Online]. Available: <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [9] K. Yadav and R. Thareja, “Comparing the performance of naive bayes and decision tree classification using r,” *International Journal of Intelligent Systems and Applications*, vol. 11, no. 12, pp. 11–19, 2019.
- [10] V. Sheth, U. Tripathi, and A. Sharma, “A comparative analysis of machine learning algorithms for classification purpose,” *Procedia Computer Science*, vol. 215, pp. 422–431, 2022.

- [11] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in python,” 2011. [Online]. Available: <https://scikit-learn.org>

## APPENDIX A: Sample of the Sensor Dataset

A representative sample of the 8000-row `sensor_dataset.csv` used for training and evaluating the machine learning models is shown below.

Table 5.1: Sample rows from `sensor_dataset.csv`

| <b>count</b> | <b>co2_level</b> | <b>ldr_diff</b> | <b>temp</b> | <b>humidity</b> | <b>fan_speed</b> | <b>light_intensity</b> |
|--------------|------------------|-----------------|-------------|-----------------|------------------|------------------------|
| 17           | 2768             | 158             | 42.5        | 60.9            | 3                | 4                      |
| 8            | 1427             | 63              | 31.4        | 69.2            | 2                | 3                      |
| 6            | 1311             | 38              | 35.2        | 65.0            | 2                | 3                      |
| 19           | 2956             | 337             | 30.4        | 63.2            | 3                | 4                      |
| 7            | 1301             | 140             | 22.8        | 61.1            | 1                | 3                      |

The fan-speed model uses `count`, `co2_level`, `temp`, and `humidity`, while the light-intensity model uses `count`, `ldr_diff`, and `co2_level`.

## APPENDIX B: On-Device Decision Tree Model

The trained Decision Tree models were converted into C++ header files and included directly in the ESP32 firmware. The following is a small excerpt from the `predict()` function of the fan-speed model.

```
int predict(float *x) {
    if (x[1] <= 1905.5) {
        if (x[1] <= 1220.5) {
            if (x[0] <= 0.5) {
                if (x[2] <= 24.80) {
                    ...
                    return 0;
                }
                ...
            }
            ...
        }
        ...
    }
}
```

*(x[0] = count, x[1] = co2\_level, x[2] = temp, x[3] = humidity)*

The light-intensity model is implemented in the same way, using `count`, `co2_level`, and `ldr_diff` as its input features. The complete generated model files are included with the project source code.

## APPENDIX C: MQTT JSON Payload Schema

**Published by ESP32 on topic `esp32/sensordata`:**

```
{  
  "count": 3,  
  "temp": 28.4,  
  "humidity": 61.2,  
  "ldr": 210,  
  "co2": 950,  
  "fan_speed": 50,  
  "light_intensity": 75,  
  "fan_manual": false,  
  "light_manual": false  
}
```

**Received by ESP32 on topic `esp32/control`:**

```
{ "fan": 70 }  
{ "light": 40 }  
{ "fan_mode": "manual" }  
{ "light_mode": "auto" }
```

The ESP32 uses the received commands to apply manual control or return the corresponding actuator to automatic operation.

## APPENDIX D: Hardware Pin Map

Table 5.2: ESP32 GPIO assignment

| Signal                 | GPIO Pin |
|------------------------|----------|
| IR Sensor 1            | GPIO 23  |
| IR Sensor 2            | GPIO 22  |
| DHT11 Data             | GPIO 4   |
| LDR (inside)           | GPIO 34  |
| LDR (outside)          | GPIO 36  |
| Gas / CO2-proxy sensor | GPIO 35  |

The two LDR readings are used to calculate `ldr_diff`, which is used as an input to the light-intensity model.